

模型能力

工具流式输出

 Copy page

💡 流式工具调用（Stream Tool Call）是 Z.ai 最新模型的特性，允许在工具调用过程中实时获取推理过程、回答内容和工具调用信息，提供更好的用户体验和实时反馈。

功能特性

工具调用在最新 GLM-5 GLM-4.7 GLM-4.6 模型中现在支持开启响应的流式输出。这允许开发者在调用 `chat.completions` 时，在不进行缓冲或JSON验证的情况下流式传输工具使用参数，从而减少调用延迟，提供更好的用户体验。

核心参数说明

- `stream=True`：启用流式输出，必须设置为 `True`
- `tool_stream=True`：启用工具调用流式输出
- `model`：使用支持工具调用的模型，仅限 `glm-4.7` `glm-4.6`

响应参数说明

流式响应中的 `delta` 对象包含以下字段：

- `reasoning_content`：模型推理过程的文本内容
- `content`：模型回答的文本内容
- `tool_calls`：工具调用信息，包含函数名和参数

通过设置 `tool_stream=True` 参数，可以启用流式工具调用功能：

>

Python SDK

安装 SDK

```
# 安装最新版本
pip install zai-sdk

# 或指定版本
pip install zai-sdk==0.2.2
```

验证安装

```
import zai
print(zai.__version__)
```

完整示例



初始化客户端

```
client = ZhipuAiClient(api_key='您的apikey')
```

创建流式工具调用请求

```
response = client.chat.completions.create(  
    model="glm-4.7", # 使用支持工具调用的模型  
    messages=[  
        {"role": "user", "content": "北京天气怎么样"},  
    ],  
    tools=[  
        {  
            "type": "function",  
            "function": {  
                "name": "get_weather",  
                "description": "获取指定地点当前的天气情况",  
                "parameters": {  
                    "type": "object",  
                    "properties": {  
                        "location": {"type": "string", "description": "城市, 例如:"},  
                        "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]},  
                    },  
                    "required": ["location"]  
                }  
            }  
        }  
    ],  
    stream=True, # 启用流式输出  
    tool_stream=True # 启用工具调用流式输出  
)
```

初始化变量用于收集流式数据

```
reasoning_content = "" # 推理过程内容
```

```
content = "" # 回答内容
```

```
final_tool_calls = {} # 工具调用信息
```

```
reasoning_started = False # 推理过程开始标志
```

```
content_started = False # 内容输出开始标志
```

处理流式响应

```
for chunk in response:
    if not chunk.choices:
        continue

    >
    delta = chunk.choices[0].delta

# 处理流式推理过程输出
if hasattr(delta, 'reasoning_content') and delta.reasoning_content:
    if not reasoning_started and delta.reasoning_content.strip():
        print("\n🧠 思考过程: ")
        reasoning_started = True
    reasoning_content += delta.reasoning_content
    print(delta.reasoning_content, end="", flush=True)

# 处理流式回答内容输出
if hasattr(delta, 'content') and delta.content:
    if not content_started and delta.content.strip():
        print("\n💬 回答内容: ")
        content_started = True
    content += delta.content
    print(delta.content, end="", flush=True)

# 处理流式工具调用信息
if delta.tool_calls:
    for tool_call in delta.tool_calls:
        index = tool_call.index
        if index not in final_tool_calls:
            # 新的工具调用
            final_tool_calls[index] = tool_call
            final_tool_calls[index].function.arguments = tool_call.function.
        else:
            # 追加工具调用参数（流式构建）
            final_tool_calls[index].function.arguments += tool_call.function

# 输出最终的工具调用信息
if final_tool_calls:
    print("\n📋 命中 Function Calls :")
    for index, tool_call in final_tool_calls.items():
        print(f" {index}: 函数名: {tool_call.function.name}, 参数: {tool_call.fur
```



智能客服系统

- 实时显示查询进度
- 改善等待体验



代码助手

- 实时代码分析过程
- 显示工具调用链